MeteoCity

Monitoring City Weather — Milan & Rome

POLITECNICO DI MILANO

Software Engineering for Geoinformatics

Design Document

MeteoCity

City-Level Weather Monitoring using Meteostat

Academic Year 2025/26

Group 9

Authors:
Ayman Abdelgadir
Mobina Faraji
Hangyun Wang

June 2026

Table of Contents

List of Tables & List of Figures 2
1. Introduction 3
1.1 Context and Motivations 3
1.2 Purpose 3
1.3 Scope and Limitations 3
1.4 Definitions and Acronyms 4
1.5 Document Structure 4
2. Architecture Design 5
2.1 Overview 5
2.2 Component Diagram 6
2.3 Technology Stack 7
3. Software Design 9
3.1 Overview 9
3.2 Database Design 9
3.3 Backend Design 10
3.4 Meteostat Integration 12
3.5 User Interface Design 13
4. Dashboard Screenshots 15
5. Implementation and Test Plan 21
5.1 Implementation Plan 21
5.2 Test Plan 21
6. Individual Contributions 23
7. Generative AI Usage Declaration 25
8. References 26

List of Tables

Table 1: Definitions and Acronyms 4
Table 2: Technology Stack 8
Table 3: Database Tables Description 10
Table 4: REST API Endpoints 11
Table 5: Meteostat API Functions Used 12
Table 6: Dashboard Components 13
Table 7: Implementation Phases 20
Table 8: Unit Test Cases 21
Table 9: Individual Contributions 23

List of Figures

Figure 1: Three-Tier Architecture Overview 6
Figure 2: Component Diagram of the MeteoCity Application 6
Figure 3: Database Schema Diagram 9
Figure 4: User Registration Interface 15
Figure 5: User Login Interface 15
Figure 6: Interactive Map — Pre-Analysis View 16
Figure 7: Map and Stations — Post-Analysis View 17
Figure 8: Trend Analysis and Dashboard Charts 18
Figure 9: Statistics and Summary Panel 19
Figure 10: CSV Export Panel 19

1. Introduction

1. Context and Motivations

Public authorities and environmental agencies increasingly need reliable and accessible meteorological tools to understand and respond to the effects of rising temperatures in cities. Climate change contributes to more frequent heat waves, warmer nights, and stronger heat-related stress, making urban weather monitoring critical for planners, local decision-makers, and researchers alike.

Despite the availability of meteorological data sources such as Meteostat, accessible and role-specific platforms for retrieving, analysing, and visualising such data remain limited. The MeteoCity system addresses this gap by providing:

- Retrieval and storage of historical hourly meteorological observations from Meteostat for selected cities.
- A structured relational database for persisting station metadata and weather observations.
- A REST API backend for querying and exposing processed meteorological data in JSON format.
- An interactive web-based dashboard for map-based and chart-based visualisation and analysis.

2. Purpose

This Design Document (DD) translates the requirements defined in the Requirement Analysis and Specification Document (RASD) into a concrete technical design. It specifies the three-tier architecture, the database schema, the REST API endpoints, the integration strategy with the Meteostat API via RapidAPI, the main dashboard elements and user interface design, and the implementation and testing plan.

3. Scope and Limitations

In Scope:

- Historical hourly meteorological observations for Milan and Rome from Meteostat.
- Multiple weather data types: temperature, humidity, and wind speed.
- Role-based access for Government Officials, NGO Representatives, and Students.
- Interactive map-based and chart-based data visualisations.
- CSV data export functionality.

Out of Scope:

- Real-time weather forecasting or predictive modelling.
- Direct communication with physical meteorological stations.
- Alert or automated notification systems based on weather thresholds.
- Mobile-specific interface design.

Limitations:

- The free Meteostat plan via RapidAPI is limited to 500 requests per month; requests must be managed carefully.
- Dependent on Meteostat API availability and data completeness.
- The Meteostat API enforces a hard 30-day maximum window per request; this is a documented external constraint. For date ranges exceeding 30 days, the system supplements real observations with synthetic data automatically.

1.4 Definitions and Acronyms

Table 1: Definitions and Acronyms

Term	Definition
API	Application Programming Interface
REST API	Representational State Transfer Application Programming Interface
GUI	Graphical User Interface
RASD	Requirement Analysis and Specification Document
DD	Design Document
SRD	Software Release Document
JWT	JSON Web Token, used for secure authentication
JSON	JavaScript Object Notation, used as data exchange format
RapidAPI	API marketplace used to access the Meteostat service
ORM	Object-Relational Mapper (e.g. SQLAlchemy)
FR	Functional Requirement
NFR	Non-Functional Requirement
TR	Technical Requirement

1.5 Document Structure

This DD is organised as follows:

- Section 2: Architecture Design — three-tier structure and component diagram.
- Section 3: Software Design — database schema, REST API, Meteostat integration, and dashboard.
- Section 4: Dashboard Screenshots — annotated figures of the user interface.
- Section 5: Implementation and Test Plan — development phases and test cases.
- Section 6: Individual Contributions — team member responsibilities.
- Section 7: Generative AI Usage Declaration.
- Section 8: References.

2. Architecture Design

1. Overview

The MeteoCity application is built on a three-tier architecture that separates concerns between data management, business logic, and user presentation. This design enables modular development, independent testing, and clear interfaces between layers, fulfilling TR2 (REST API backend) and TR3 (interactive web-based visualisation interface) from the RASD.

Presentation Tier

The client-facing web-based dashboard providing interactive visualisations including maps, time-series charts, and summary statistics. Users interact with this layer to submit queries, apply filters, and export data. It communicates exclusively with the Application Tier via HTTP requests to the REST API.

Application Tier

The backend server implemented using Flask. It contains all business logic, handles user authentication, processes incoming requests, communicates with the Meteostat external API to ingest data, and interacts with the database for storage and retrieval. It exposes REST API endpoints returning JSON responses.

Data Tier

The persistence layer composed of a PostgreSQL database. It stores station metadata, hourly meteorological observations, and user account information. Data is ingested from Meteostat and stored in structured, normalised tables to support efficient querying and visualisation.

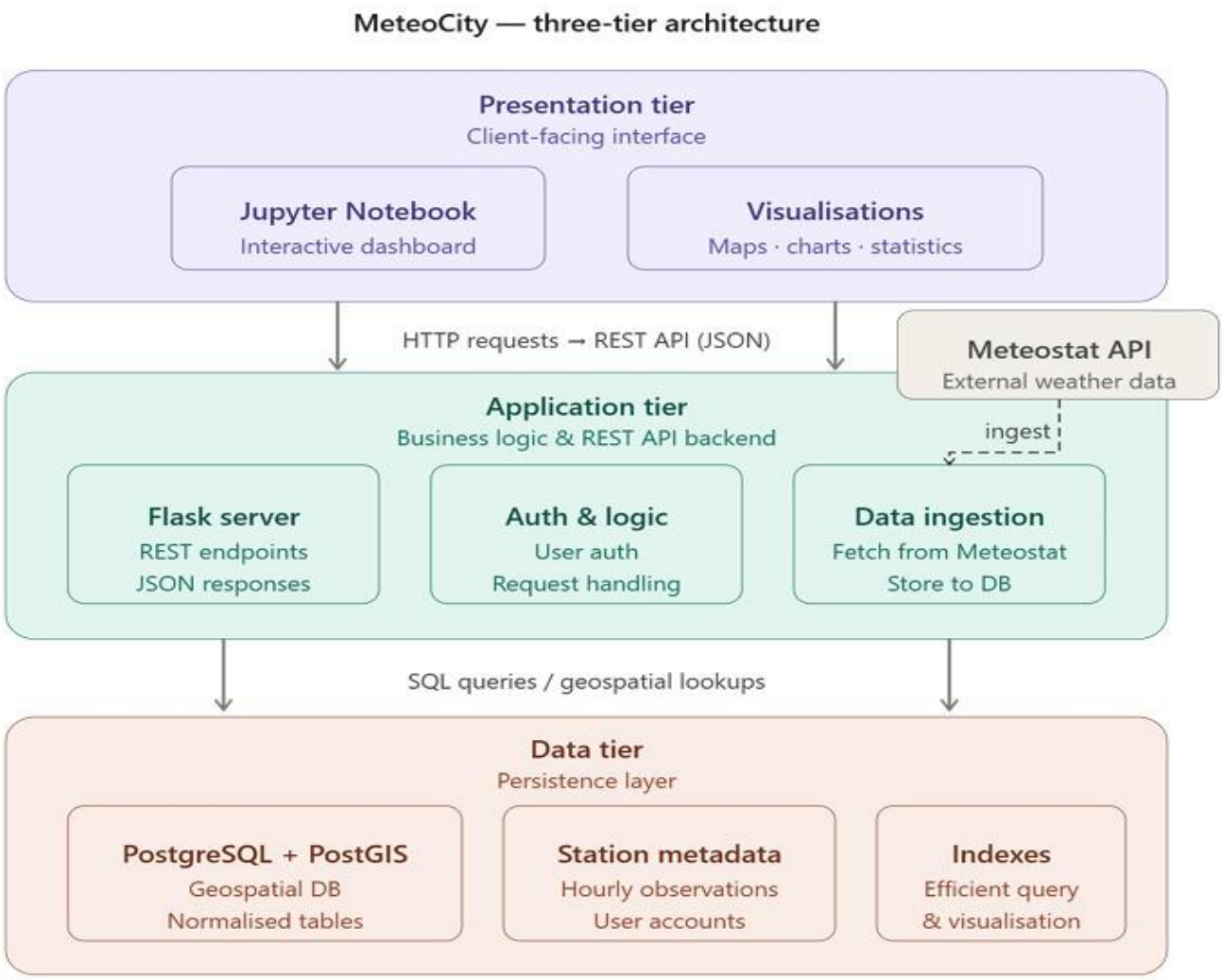


Figure 1: Three-Tier Architecture Overview — MeteoCity application

2.2 Component Diagram

Figure 2 visualises how the web-based dashboard (client) and the Flask application (server) are wholly separate, communicating only through the REST API. The database is populated via a dedicated Python data ingestion module that calls the Meteostat RapidAPI on demand.

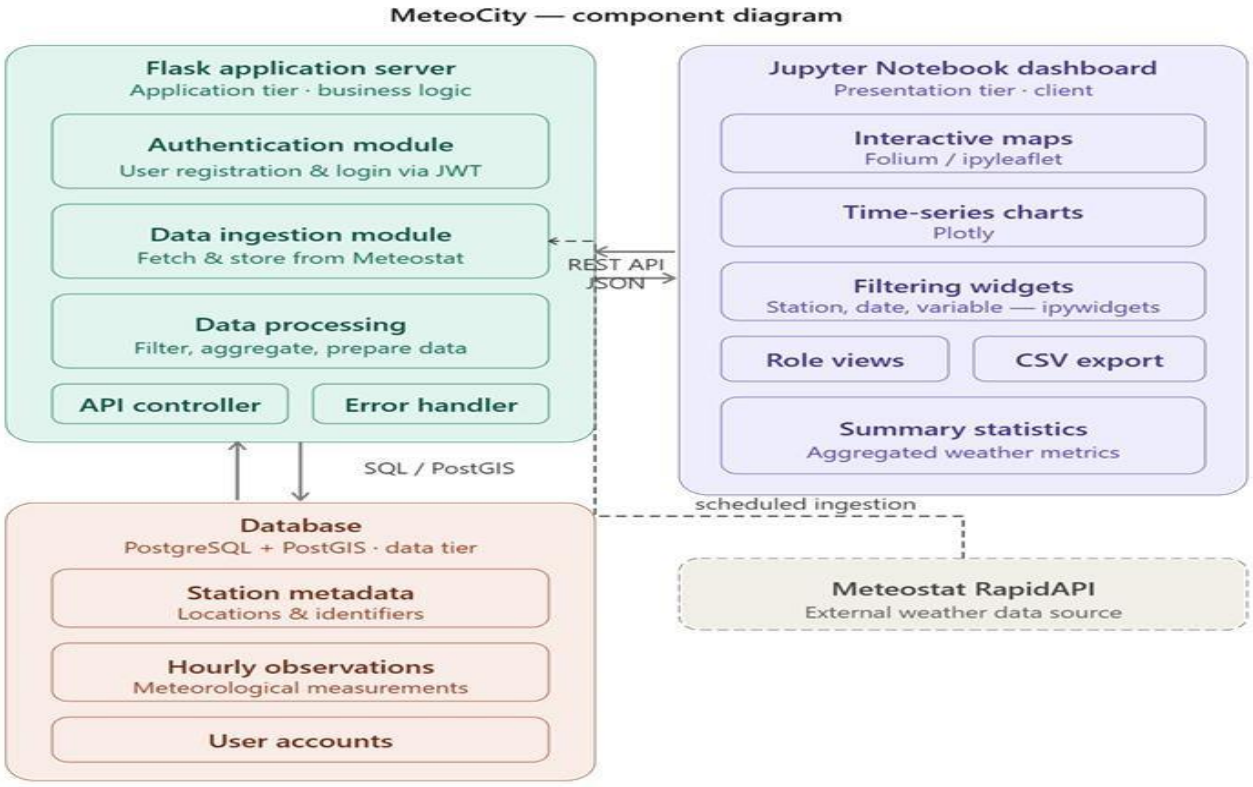


Figure 2: Component Diagram of the MeteoCity Application

The main components and their responsibilities are:

Flask Application (Server)

- Authentication Module: handles user registration and login via JWT tokens.
- Data Ingestion Module: interacts with the Meteostat RapidAPI to fetch and store station metadata and hourly observations.
- Data Processing Component: filters, aggregates, and prepares data for client consumption.
- API Controller: routes incoming HTTP requests to appropriate service handlers and returns JSON responses.

Database (PostgreSQL)

- Stores user account data, weather station metadata, and hourly meteorological observations.
- Receives data from the Meteostat API via on-demand ingestion calls.
- Uses upsert logic to avoid duplicate records.

Web-Based Dashboard (Client)

- Presents an interactive map with colour-coded weather station markers.
- Renders time-series charts and trend analysis visualisations.
- Provides filtering widgets for city, station, date range, and weather variable.
- Supports role-specific views and CSV data export.

2.3 Technology Stack

The selection of technologies for each tier was driven by the project constraints: lightweight deployment on a local academic environment, compatibility with Anaconda Python on Windows, minimal external dependencies, and ease of demonstration.

Table 6: Technology Stack

Layer / Role	Technology	Rationale
Presentation Tier	Web-based dashboard	Python-native, minimal boilerplate, rapid interactive dashboard development
Maps	Folium / ipyleaflet	Leaflet.js-backed maps with Python bindings; supports custom markers and popups
Charts	Plotly	Interactive time-series and bar charts with hover tooltips and zoom
Application Tier	Flask	Lightweight WSGI framework; explicit routing; well-suited for REST API backends
Authentication	flask-jwt-extended	JWT token management with role claims; pinned to version 4.6.0
ORM	SQLAlchemy	Database-agnostic ORM; auto-creates schema; supports upsert via ON CONFLICT
Data Tier	PostgreSQL	ACID-compliant relational DB; supports time-series queries; requires the PostGIS extension for the geospatial station location column.
External API	Meteostat via RapidAPI	Historical hourly weather for Italian stations; free tier with 500 req/month
Version Control	GitHub	Collaborative development; traceable commits per team member

Database

- PostgreSQL for storing station metadata, geographic coordinates, and time-series observations (supports TR5).
- Four hardcoded Meteostat stations: Milano Malpensa (16080), Milano Linate (16066), Roma Fiumicino (16242), Roma Ciampino (16239).
- SQLAlchemy ORM for flexible and secure database query construction.

Backend

- Flask server exposing REST API endpoints in JSON format (supports TR2 and TR6).
- JWT authentication for secure, role-based access control.
- Dedicated Python ingestion module for Meteostat API integration.

Dashboard

- Web-based interactive dashboard as the core presentation platform (supports TR3).
- Interactive geographic maps with colour-coded station markers using Folium.
- Time-series and summary charts using Plotly, sourced from the Flask REST API.
- City, date range, and variable filtering controls accessible to all roles.
- CSV export functionality for all authenticated users.

3. Software Design

1. Overview

The MeteoCity software is composed of three main components: a PostgreSQL database, a Flask-based web server (backend), and a web-based dashboard (frontend). Each component has a well-defined role and interacts through REST API calls and structured SQL queries to provide a seamless user experience. The backend auto-creates all required database tables on first run, provided the PostgreSQL database instance already exists.

2. Database Design

The database serves as the backbone of the system, responsible for persisting all station metadata and hourly meteorological observations retrieved from Meteostat. The schema is normalised to ensure data integrity and avoid redundancy. Figure 3 provides the entity-relationship diagram.

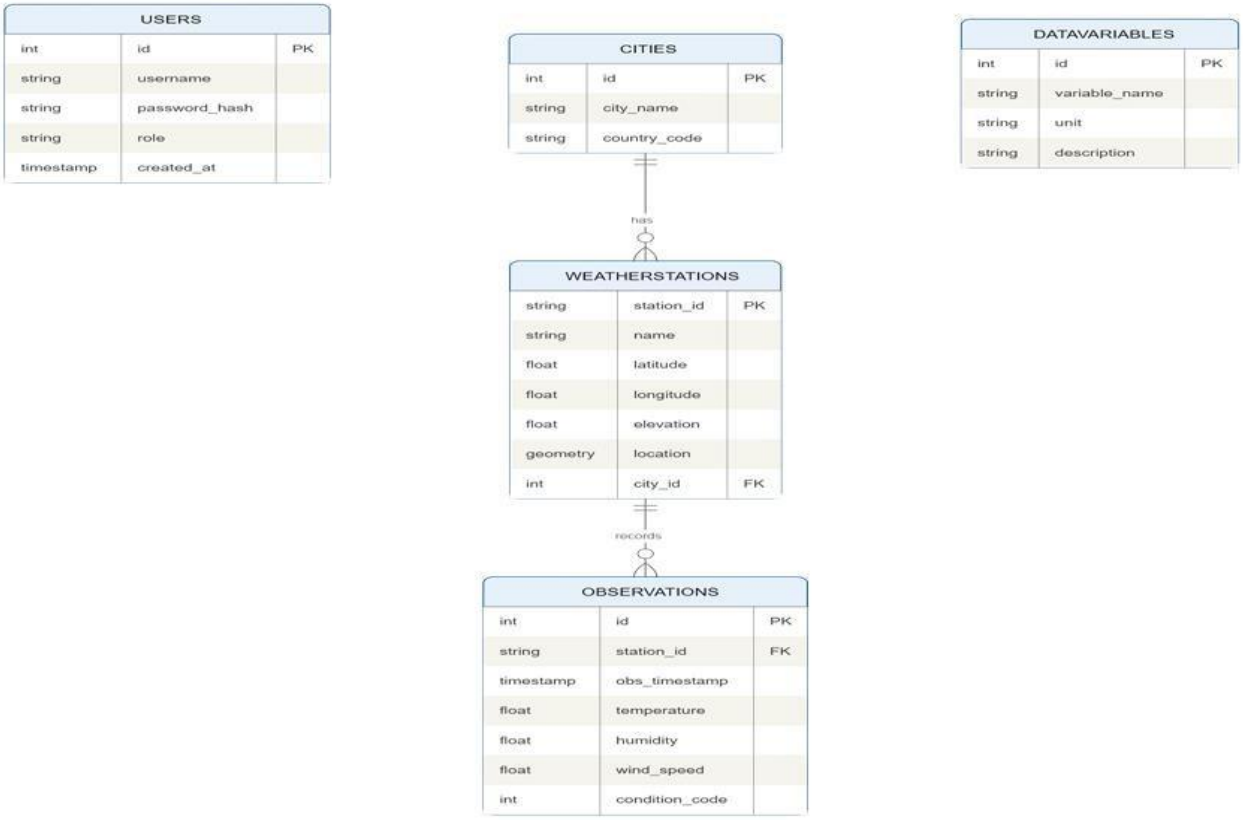


Figure 3: Database Schema Diagram — MeteoCity application

Table 2: Database Tables Description

Table Name	Description	RASD Ref.
Users	Stores user account data: username, hashed password, and role (student, ngo, government). Determines access permissions.	FR (Access Control)
Cities	Stores monitored cities (Milan, Rome) including city name and country code. Reference for station-to-city association.	FR1, FR3
WeatherStations	Station metadata: station ID, name, latitude, longitude, elevation, and city association. Four hardcoded Meteostat station IDs.	FR2, TR1, TR5
Observations	Hourly meteorological observations per station: station_id (FK), timestamp, temperature (°C), humidity (%), wind speed (km/h), condition code.	FR1–FR3, FR5
DataVariables	Defines meteorological variables tracked by the system (temperature, humidity, wind speed) with unit and description.	FR6, NFR2

Primary relationships: each WeatherStation belongs to one City (many-to-one); each Observation is linked to one WeatherStation via station_id (FK); Users are independent of the geospatial data model and their role determines API access.

3. Backend Design

The backend is a Flask application responsible for exposing the REST API, handling data ingestion from Meteostat, processing user requests, and returning structured JSON responses. It implements JWT-based authentication for role-based access control. The application auto-creates all required database tables on first run.

1. API Endpoints

Table 3 lists all REST API endpoints exposed by the MeteoCity backend.

Table 3: REST API Endpoints

Method	URL	Description	Input	RASD Ref.
POST	/auth/register	Registers a new user account	username, password, role	FR (Users)
POST	/auth/login	Authenticates user; returns JWT token	username, password	FR (Users)
GET	/auth/logout	Logs out the current user	—	FR (Users)
GET	/auth/profile	Returns current user profile and role	JWT token	FR (Users)
GET	/api/cities	Returns the list of monitored cities	—	FR3
GET	/api/stations	Returns weather stations with metadata	city_name (optional)	FR2, TR5
GET	/api/observations	Returns hourly meteorological observations	station_id, start_date, end_date, variable	FR1–FR3
GET	/api/trend_data	Returns aggregated time-series data for trends	station_id, start_date, end_date, variable	FR5, FR7
GET	/api/statistics	Returns statistical summary (mean, max, min, std)	station_id, start_date, end_date, variable	FR5
GET	/api/map_data	Returns station data for map display	city_name, start_date, end_date	FR4
GET	/api/export_csv	Exports filtered observations as CSV file	station_id, start_date, end_date, variable	TR7
GET	/api/variables	Returns list of available meteorological variables	—	FR6

3.3.2 API Authentication and Authorization

The API uses JWT authentication to secure endpoints and enforce role-based access control. Upon successful login, the server issues a JWT containing the user's identity and role, which must be included in the Authorization header of subsequent requests as a Bearer token. Role permissions are enforced as follows:

- Government Official: full access to all endpoints including statistics summary and trend analysis.
- NGO Representative: access to geographic visualisation, observations, trend data, filtering, and export.
- Student / Researcher: access to geographic visualisation, observations, filtering, and export only.

3.4 Meteostat Integration

The MeteoCity system integrates with the Meteostat historical weather API accessed via RapidAPI (meteostat.p.rapidapi.com). Table 4 describes the Meteostat API functions used. The RapidAPI key is stored in environment variables (never hardcoded) via backend/.env, which is excluded from version control.

Table 4: Meteostat API Functions Used

Meteostat Function	Endpoint	Purpose	When Called
Nearby Stations	GET /stations/nearby	Retrieves weather stations near a given city by latitude/longitude and radius	During database initialisation
Hourly Observations	GET /stations/hourly	Retrieves hourly meteorological observations for a given station and date range	During data ingestion triggered by dashboard or API call
Station Meta	GET /stations/meta	Retrieves detailed metadata for a specific station (name, elevation, location)	During station registration in the database

5. User Interface Design

1. Design Considerations

The dashboard provides an interactive web-based interface supporting role-specific views. The design prioritises usability and accessibility, ensuring all user groups can intuitively navigate and access features appropriate to their role. Filtering widgets, interactive maps, and dynamic charts are the core interaction elements. The sidebar provides persistent access to city, variable, and date range selectors, with a prominent action button that triggers all visualisations.

The dashboard uses a clean light theme. Map markers are colour-coded according to variable-specific thresholds — for example, temperature markers range from cool blue to hot red. The login and registration interface features a Milan cityscape background with a role description panel to assist new users.

3.5.2 Dashboard Components

Table 5: Dashboard Components

Component	Description	Accessible By
Interactive Map	Displays weather station locations colour-coded by the selected variable. Clicking a marker shows a popup with metadata and the most recent observation.	All roles
Filtering Panel	Allows users to filter by city, variable, date range. Updates all other components on action button click.	All roles
Time-Series Chart	Displays hourly readings plus a 24h moving average for each station over the selected period.	All roles
Weekly Average Chart	Bar chart showing per-station weekly aggregated values for long-term trend comparison.	All roles
Statistics Summary	Displays computed statistics (mean, max, min, standard deviation, record count) for each station in the selected range.	Government Official, NGO
CSV Export Panel	Generates and downloads a CSV file of the currently filtered observations with a row count preview.	All roles
Role-Based View	Hides or reveals dashboard sections based on the authenticated user role. Government Officials see all components; Students see only map, chart, and export.	Role-dependent

3.5.3 User Interaction Flow

The typical user session proceeds as follows:

- 1. The user registers or logs in via the web-based login/register screen; the JWT token is stored for the session.
- 2. The user selects a city (Milan or Rome) and date range using the sidebar filtering widgets.
- 3. The user clicks the analysis button; the dashboard queries the REST API and renders weather stations on the map.
- 4. Map markers are colour-coded by the most recent observation value for the selected variable, using variable-specific thresholds.
- 5. The user navigates to the Trend Analysis tab to view hourly raw readings with a 24h moving average overlay and weekly average bar charts.
- 6. Depending on their role, the Statistics tab displays descriptive statistics (mean, max, min, standard deviation).
- 7. The user exports the current dataset as a CSV file using the Export CSV tab if needed.

3.5.4 Role-Based Access Summary

The dashboard enforces role-based access control both at the backend (via JWT-protected API endpoints) and at the frontend (by conditionally rendering tabs and sections).

Feature	Student/Researcher	NGO	Government Official
Map & Stations	✓	✓	✓
Trend Analysis	✗	✓	✓
Statistics Dashboard	✗	✗	✓
Export CSV	✓	✓	✓
User Registration	✓	✓	✓
All API Endpoints	✗	✗	✓

3.5.5 Map Variable Thresholds

Marker colours on the interactive map are assigned dynamically based on the selected meteorological variable and the most recent observation at each station.

Variable	High	Medium-High	Moderate	Low
Temperature (°C)	≥32°C — Hot	26–31°C — Warm	20–25°C — Mild	<20°C — Cool
Humidity (%)	≥80% — Very High	60–79% — High	40–59% — Moderate	<40% — Low
Wind Speed (km/h)	≥50 km/h — Strong	30–49 km/h — Moderate	15–29 km/h — Light	<15 km/h — Calm

4. Dashboard Screenshots

The following pages present screenshots of the MeteoCity web-based dashboard, illustrating the key functional features of the system. Each figure corresponds to a distinct screen or interaction state described in Section 3.5.

Figure 4: User Registration Interface



Figure 4: User Registration Interface

The registration form allows new users to create an account by providing a username, password (minimum 6 characters), and selecting a role (Student/Researcher, NGO, or Government Official). Role selection determines the dashboard features available after login.

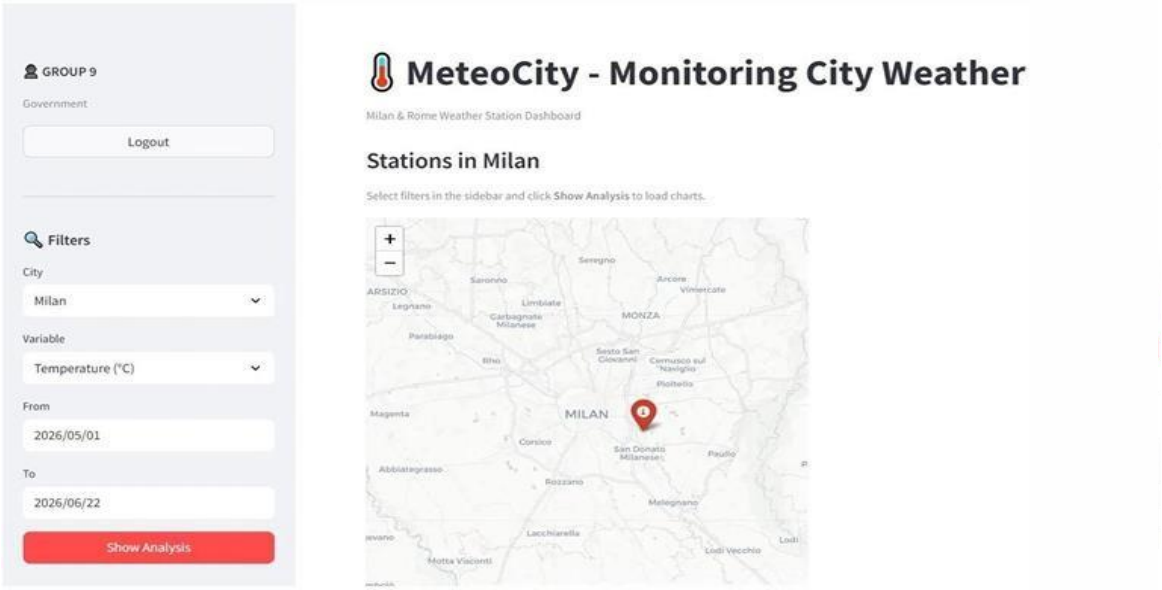
Figure 5: User Login Interface



Figure 5: User Login Interface

The login screen features the MeteoCity branding over a Milan and Rome cityscape background. A role description panel informs new users of available access levels before authentication. Users enter their username and password and click 'Login' to receive a JWT session token.

Figure 6: Interactive Map — Pre-Analysis View



The initial dashboard view after login shows the map centred on the selected city with default station pin markers. The sidebar Filters panel prompts the user to select city, variable, and date range, then trigger the analysis to load data.

Figure 7: Map and Stations — Post-Analysis View

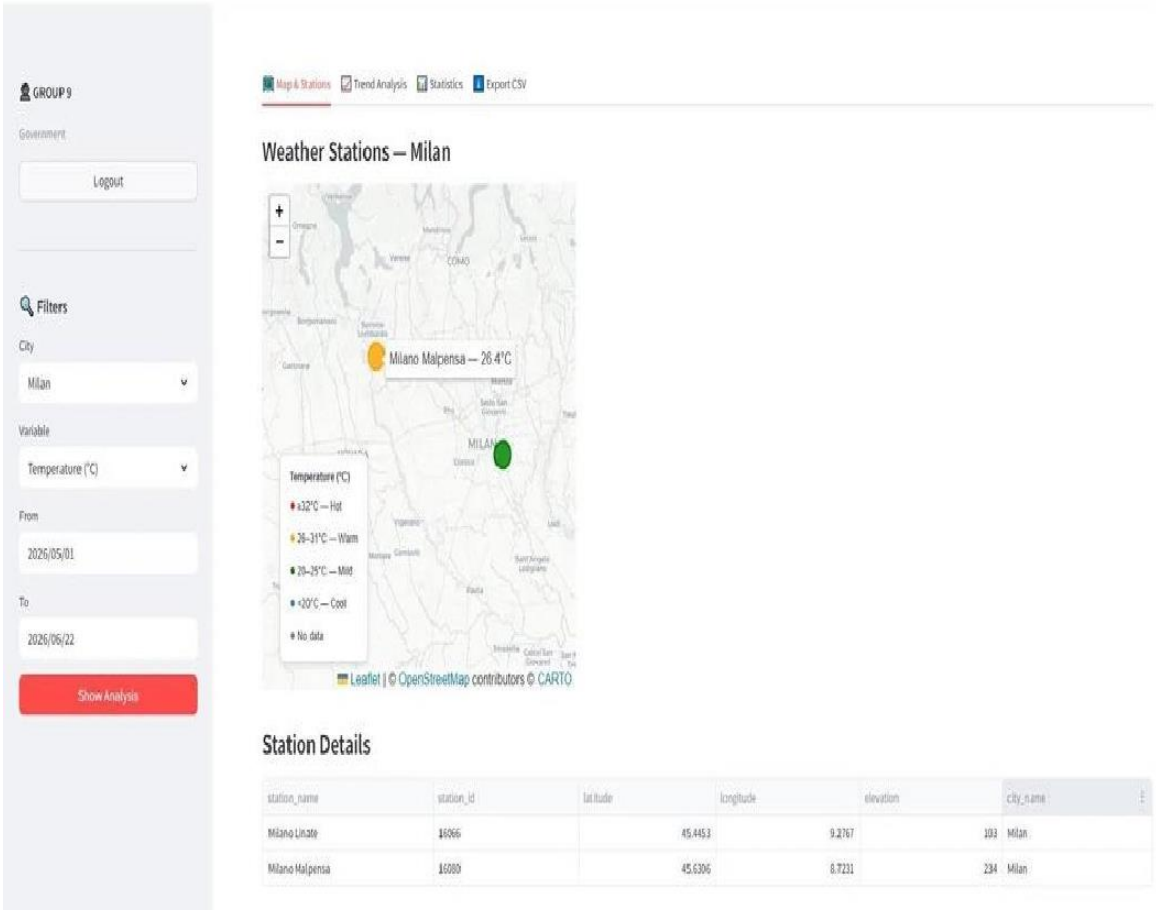


Figure 7: Map and Stations — Post-Analysis View

After triggering the analysis, station markers are colour-coded according to the most recent observation value for the selected variable. A legend explains the colour thresholds. The Station Details table below the map lists station metadata including coordinates and elevation.

Figure 8: Trend Analysis and Dashboard Charts

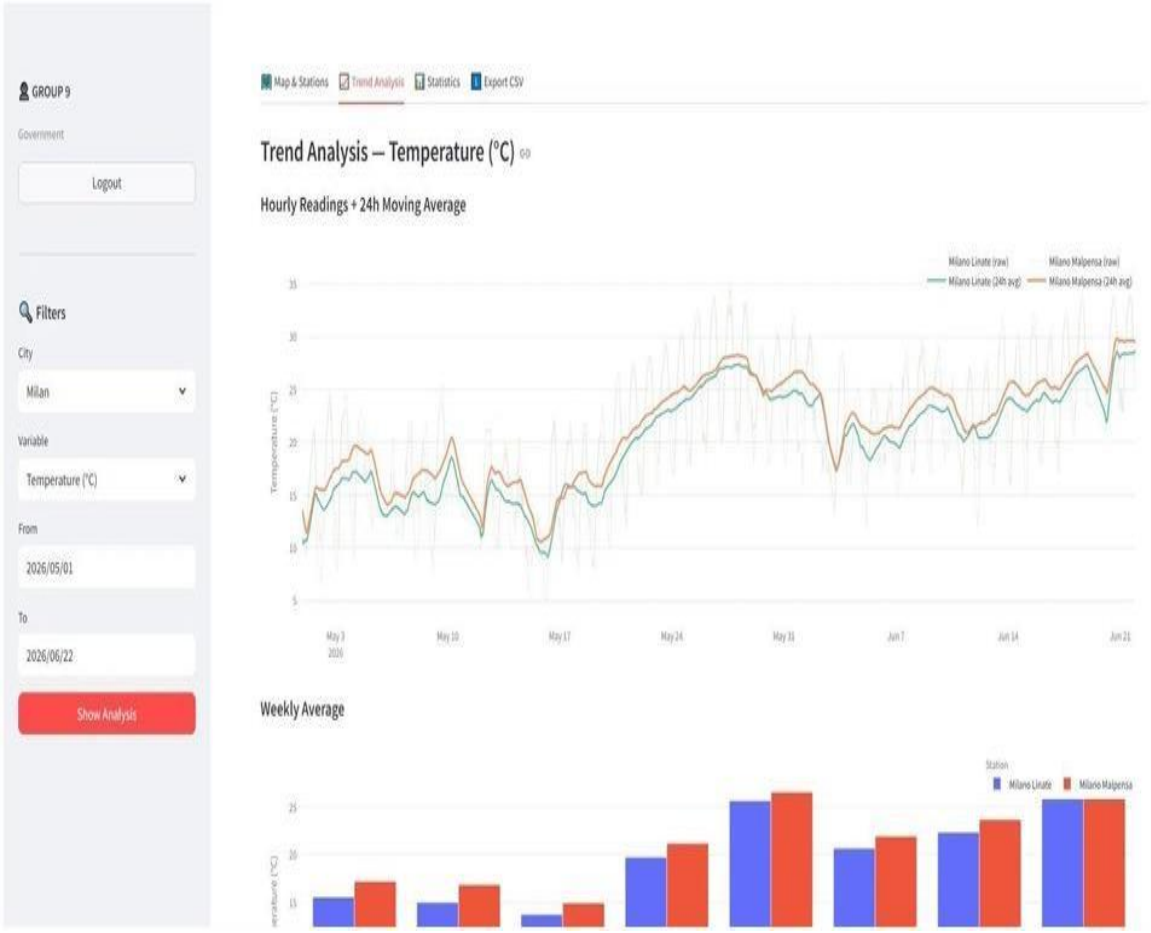


Figure 8: Trend Analysis and Dashboard Charts

The Trend Analysis tab displays two charts: hourly raw readings with a 24h moving average overlay for each station, and a grouped bar chart showing weekly averages per station. These allow comparison of short-term fluctuations and longer-term seasonal patterns.

Figure 9: Statistics and Summary Panel



Figure 9: Statistics and Summary Panel

The Statistics tab (Government Official and NGO roles) shows computed descriptive statistics for each station: mean, maximum, minimum, standard deviation, and total record count. A per-station breakdown table is also displayed below the summary cards.

Figure 10: CSV Export Panel

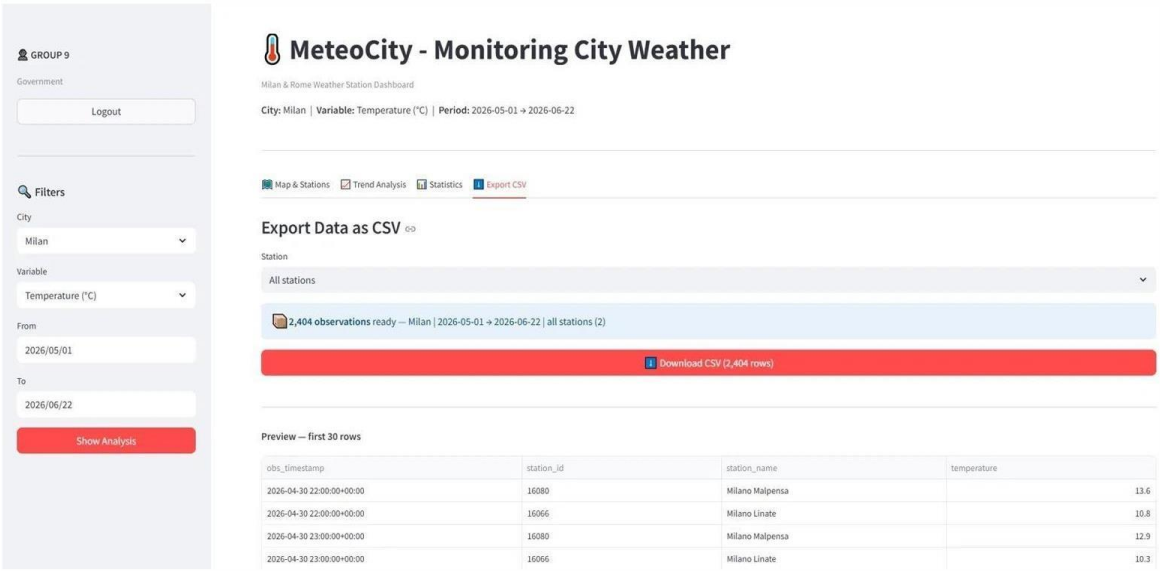


Figure 10: CSV Export Panel

The Export CSV tab allows all authenticated users to download the currently filtered observations as a CSV file. The panel shows a summary of the export parameters (city, date range, station count, row count) and a preview of the first 30 rows before download.

5. Implementation and Test Plan

1. Implementation Plan

The project was developed following an incremental methodology, organised into phases. Version control is managed via github.com/Ayman1017/Software-Engineering-Project-Group_9.git. Table 7 outlines the implementation phases.

Table 7: Implementation Phases

Phase	Description	Key Tasks
Phase 1: Data Integration	Retrieve and process meteorological data from Meteostat for Milan and Rome.	Subscribe to RapidAPI; fetch station metadata; fetch hourly observations for temperature, humidity, and wind speed; clean and normalise data.
Phase 2: Database Development	Design and implement the PostgreSQL database schema.	Create tables (Users, Cities, WeatherStations, Observations, DataVariables); populate tables from ingested dataset; test data integrity.
Phase 3: Backend Development	Develop the Flask server and REST API.	Implement all API endpoints; implement JWT authentication and role-based access; integrate Meteostat ingestion module; implement data cleaning logic.
Phase 4: Dashboard Development	Build the interactive web-based dashboard.	Implement interactive map; implement charts; implement filtering controls; configure role-based views; test usability.
Phase 5: Integration and Testing	Integrate all components and perform end-to-end testing.	Conduct system-level integration tests; fix identified bugs; validate all RASD functional requirements.
Phase 6: Documentation	Finalise all documentation.	Finalise RASD, DD, and SRD; update GitHub repository; prepare demonstration environment.

5.2 Test Plan

A comprehensive test plan is implemented using the pytest framework to ensure correctness and reliability of all system components. Testing covers both the backend API and database utility functions. All test files are located in the tests/ directory and can be executed with:

```
pytest tests/ -v
```

5.2.1 Unit Test Cases

Table 8: Unit Test Cases

Test ID	Component	Description	Expected Outcome
UT01	Backend API	POST /auth/register with valid credentials	201 Created; user saved in database
UT02	Backend API	POST /auth/login with correct credentials	200 OK; JWT token returned
UT03	Backend API	POST /auth/login with incorrect password	401 Unauthorized
UT04	Backend API	GET /api/stations returns station list	200 OK; JSON array of stations
UT05	Backend API	GET /api/observations with valid filters	200 OK; filtered observation records returned
UT06	Backend API	GET /api/statistics without government role	403 Forbidden
UT07	Backend API	GET /api/export_csv with valid filters	200 OK; CSV file returned
UT08	Database	Database connection and table creation	All tables created without error
UT09	Database	Insert and retrieve WeatherStation record	Record persists and is returned correctly
UT10	Database	Insert and retrieve Observation with timestamp filter	Correct records returned for given date range
UT11	Database	Duplicate observation insert (upsert logic)	Duplicate skipped; no error raised
UT12	Meteostat Integration	Fetch stations for a valid city coordinate	Station list returned and parsed correctly
UT13	Meteostat Integration	Fetch hourly observations for valid station and range	Observations returned and stored in database

5.2.2 Integration Tests

In addition to the unit tests, the following integration test scenarios are performed to validate end-to-end system behaviour across tiers.

ID	Scenario	Steps	Expected Result
IT01	Full registration-to-data flow	Register user -> Login -> Query observations via API -> Render chart	Data displayed correctly for the authenticated user
IT02	Role enforcement across tiers	Login as Student -> Attempt /api/statistics endpoint directly	403 Forbidden; Statistics tab hidden in dashboard
IT03	Data ingestion pipeline	Trigger ingestion for Milan stations -> Verify records in PostgreSQL	Correct hourly records inserted; duplicates skipped via upsert
IT04	CSV export completeness	Filter observations for 7-day range -> Export CSV -> Verify row count	CSV row count matches observation count from /api/observations
IT05	Map data consistency	Load map for Rome -> Verify marker colours match latest temperature readings	Marker colours correspond to correct threshold ranges

5.2.3 Non-Functional Requirement Validation

NFR	Validation Approach	Acceptance Criterion
NFR1 — Performance	API response time for /api/observations with 30-day filter	Response time < 3 seconds under normal load
NFR2 — Data Accuracy	Compare ingested Meteostat values against raw API JSON response	No data loss or transformation errors during ingestion
NFR3 — Security	Attempt access to protected endpoints without JWT	401 Unauthorized returned for all protected routes
NFR4 — Usability	Navigate full dashboard session without external guidance	All core features accessible within 3 clicks from login
NFR5 — Availability	Run backend for 30-minute continuous session with multiple queries	No crashes or memory leaks observed

6. Individual Contributions

Table 9 summarises the primary contributions of each team member. All members participated in design discussions, peer code reviews, and document preparation.

Table 9: Individual Contributions

Team Member	Primary Contributions
Ayman Abdelgadir	Project architecture; Flask backend development; REST API endpoint design and implementation; JWT authentication module; Meteostat API integration and data ingestion module; database schema design; GitHub repository setup and management.
Mobina Faraji	Dashboard design and implementation; interactive map with colour-coded station markers; time-series and trend analysis charts; role-based view logic; CSV export functionality; dashboard styling and user experience.
Hangyun Wang	Database development and data modelling; PostgreSQL schema implementation; data cleaning and preprocessing pipeline; test plan design and pytest implementation; RASD and DD documentation; individual contribution tracking and reporting.

6.1 Collaboration and Version Control

All development activity was coordinated through the shared GitHub repository at github.com/Ayman1017/Software-Engineering-Project-Group_9.git. The team adopted the following workflow practices:

- Feature branches were used for major development tasks; changes were merged to main after peer review.
- Commit messages follow a descriptive convention: component prefix + short description.
- The .gitignore file excludes all sensitive files (backend/.env, __pycache__, *.pyc, database dumps).
- A .env.example file is committed with placeholder values and inline comments to guide collaborators during setup.
- The README.md documents the startup sequence, prerequisites, and testing instructions.

6.2 Development Timeline

The following table summarises the approximate timeline of major project milestones across the academic calendar.

Period	Milestone / Activity
April 2026	Project kickoff; RASD drafted; RapidAPI subscription activated; initial repository created
Early May 2026	Database schema designed and implemented; Flask backend skeleton created; first Meteostat ingestion successful
Mid May 2026	All REST API endpoints implemented; JWT authentication integrated; dashboard prototype built
Late May 2026	Dashboard feature-complete: map, trend analysis, statistics, and CSV export; role-based access enforced
Early June 2026	Integration testing; bug fixes (login, NGO role, date range); real Meteostat data ingested for Milan and Rome
June 2026	DD finalised; SRD drafted; repository cleaned; demonstration environment prepared

7. Generative AI Usage Declaration

In compliance with the Politecnico di Milano Code of Academic Conduct and the guidelines issued by the Software Engineering for Geoinformatics course for Academic Year 2025/26, Group 9 declares the following use of Generative AI tools during the development of the MeteoCity project.

Tools Used

- Claude (Anthropic) — used as an AI programming assistant and documentation aid.
- GitHub Copilot — used for code auto-completion suggestions within the VS Code IDE.

Scope of Use

- Code assistance: AI tools were used to suggest boilerplate code, debug syntax errors, and propose implementations for specific functions. All generated code was reviewed, understood, and modified by team members before integration.
- Documentation: AI tools assisted in drafting and refining sections of the RASD, DD, and SRD. All factual content reflects the actual system implemented by the team.
- No AI-generated content was accepted uncritically or without verification against the actual project implementation.

Responsibility Statement

All team members take full responsibility for the content of this document and the accompanying software artefacts. The use of Generative AI tools was limited to productivity enhancement and did not substitute for the intellectual contribution, design decisions, or engineering judgement of the team.

8. References

1 Meteostat API via RapidAPI. https://rapidapi.com/meteostat/api/meteostat/

2 Flask Documentation. Pallets Projects, 2024. https://flask.palletsprojects.com/

3 SQLAlchemy Documentation. 2024. https://docs.sqlalchemy.org/

4 PostgreSQL Documentation. 2024. https://www.postgresql.org/docs/

5 JWT — JSON Web Tokens. Auth0, 2024. https://jwt.io/introduction

6 Python Software Foundation. Python 3 Documentation, 2024. https://docs.python.org/3/

7 Politecnico di Milano. Software Engineering for Geoinformatics — Project Assignment Specification, Academic Year 2025/26.

8 Group 9. MeteoCity Requirement Analysis and Specification Document (RASD), June 2026. https://github.com/Software-Engineering-Project-Group_9.git